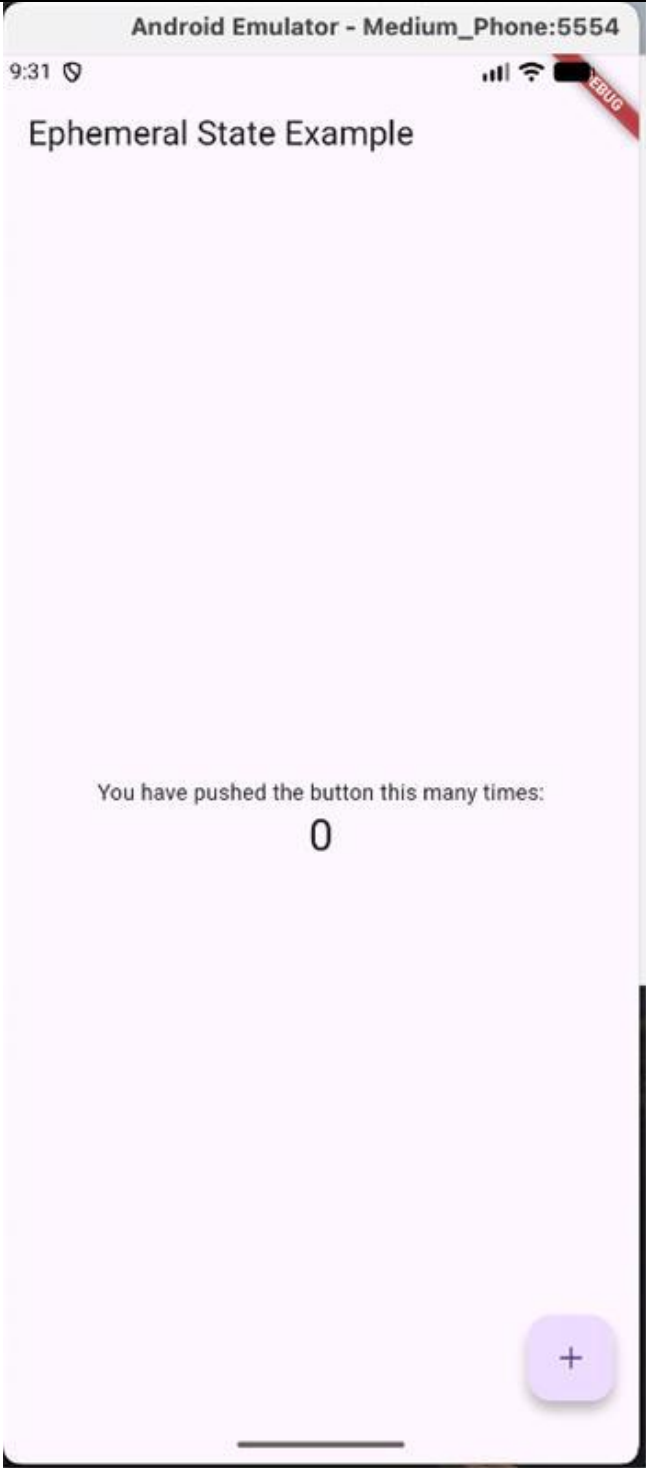


Lab Activity {*Change the title*}

Submission Form

Name	Jamaine Grace Tuazon
Section	INF231
Date	July 28, 2026

Place output based on the source code.



Place your sample code and output based on the enhancement. You may add rows or cells if needed.



main.dart

```
// Lab Activity 1 - Ephemeral vs App State
// Counter screen uses setState (ephemeral). Settings screen uses Provider for
// the light/dark theme (app state).
```

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
```

```
void main() {
  // wrap the app so ThemeModel can be reached from any screen
  runApp(
    ChangeNotifierProvider(
      create: (context) => ThemeModel(),
      child: const MyApp(),
    ),
  );
}
```

```
// holds the app state (theme). ChangeNotifier lets it tell widgets to rebuild
class ThemeModel with ChangeNotifier {
  bool _isDark = false;

  bool get isDark => _isDark;

  // switch between light and dark, then update the widgets listening
  void toggleTheme() {
    _isDark = !_isDark;
    notifyListeners();
  }
}
```

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final themeModel = Provider.of<ThemeModel>(context);

    return MaterialApp(
      title: 'Ephemeral vs App State',
      debugShowCheckedModeBanner: false,
      // pick the theme based on app state
      theme: themeModel.isDark ? ThemeData.dark() : ThemeData.light(),
      home: const CounterScreen(),
    );
  }
}
```

```
// stateful because the counter value belongs only to this screen
class CounterScreen extends StatefulWidget {
  const CounterScreen({super.key});

  @override
  State<CounterScreen> createState() => _CounterScreenState();
}
```

```
class _CounterScreenState extends State<CounterScreen> {
  int _counter = 0;

  late final ThemeModel _themeModel;

  @override
  void initState() {
    super.initState();
    // listen: false since we add our own listener below
    _themeModel = Provider.of<ThemeModel>(context, listen: false);
    _themeModel.addListener(_resetCounterOnThemeChange);
  }

  @override
  void dispose() {
    // remove listener so it doesn't leak
    _themeModel.removeListener(_resetCounterOnThemeChange);
    super.dispose();
  }
}
```

```
// reset the counter whenever the theme is toggled
void _resetCounterOnThemeChange() {
  setState(() {
    _counter = 0;
  });
}

void _incrementCounter() {
  setState(() {
    _counter++;
  });
}

// go to the theme settings screen
void _openSettings() {
  Navigator.of(context).push(
    MaterialPageRoute(builder: (context) => const SettingsScreen()),
  );
}
```



```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Counter'),
      actions: [
        // settings icon on the top right
        IconButton(
          icon: const Icon(Icons.settings),
          tooltip: 'Settings',
          onPressed: _openSettings,
        ),
      ],
    ),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          const Text('You have pushed the button this many times:'),
          Text(
            '$_counter',
            style: Theme.of(context).textTheme.headlineMedium,
          ),
        ],
      ),
    ),
    floatingActionButton: FloatingActionButton(
      onPressed: _incrementCounter,
      tooltip: 'Increment',
      child: const Icon(Icons.add),
    ),
  );
}

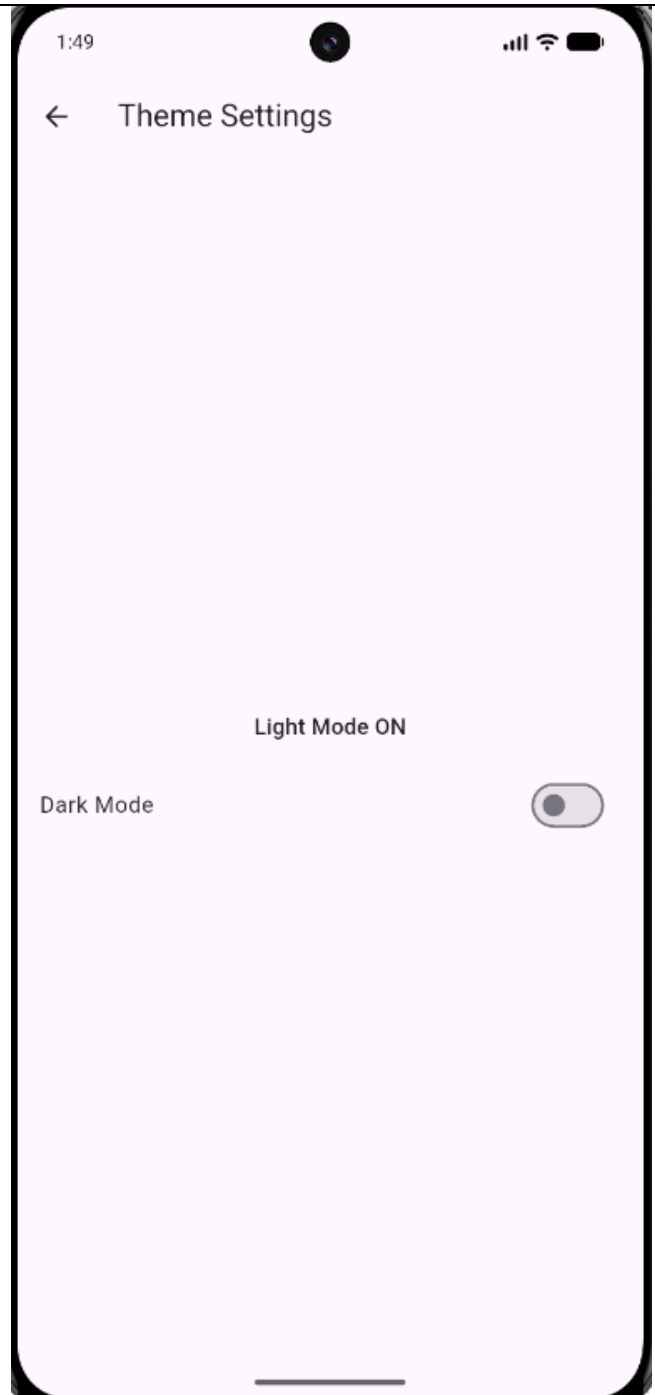
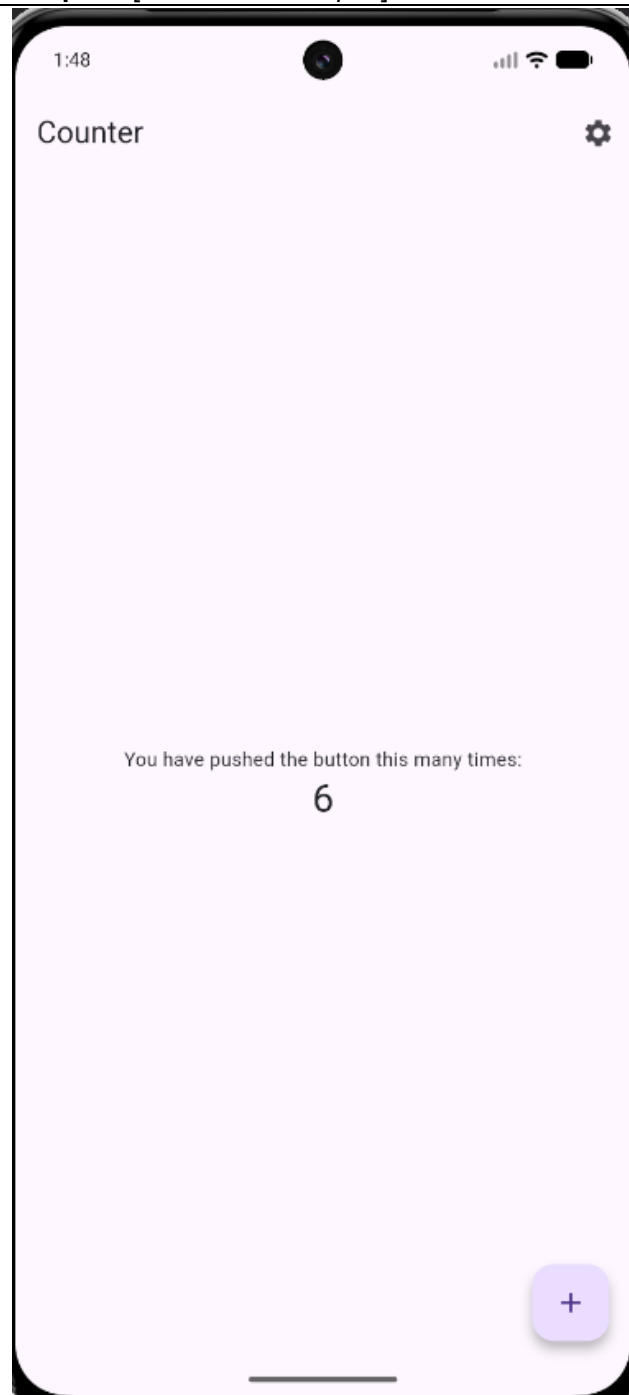
// theme toggle screen, reads and updates the shared ThemeModel
class SettingsScreen extends StatelessWidget {
  const SettingsScreen({super.key});

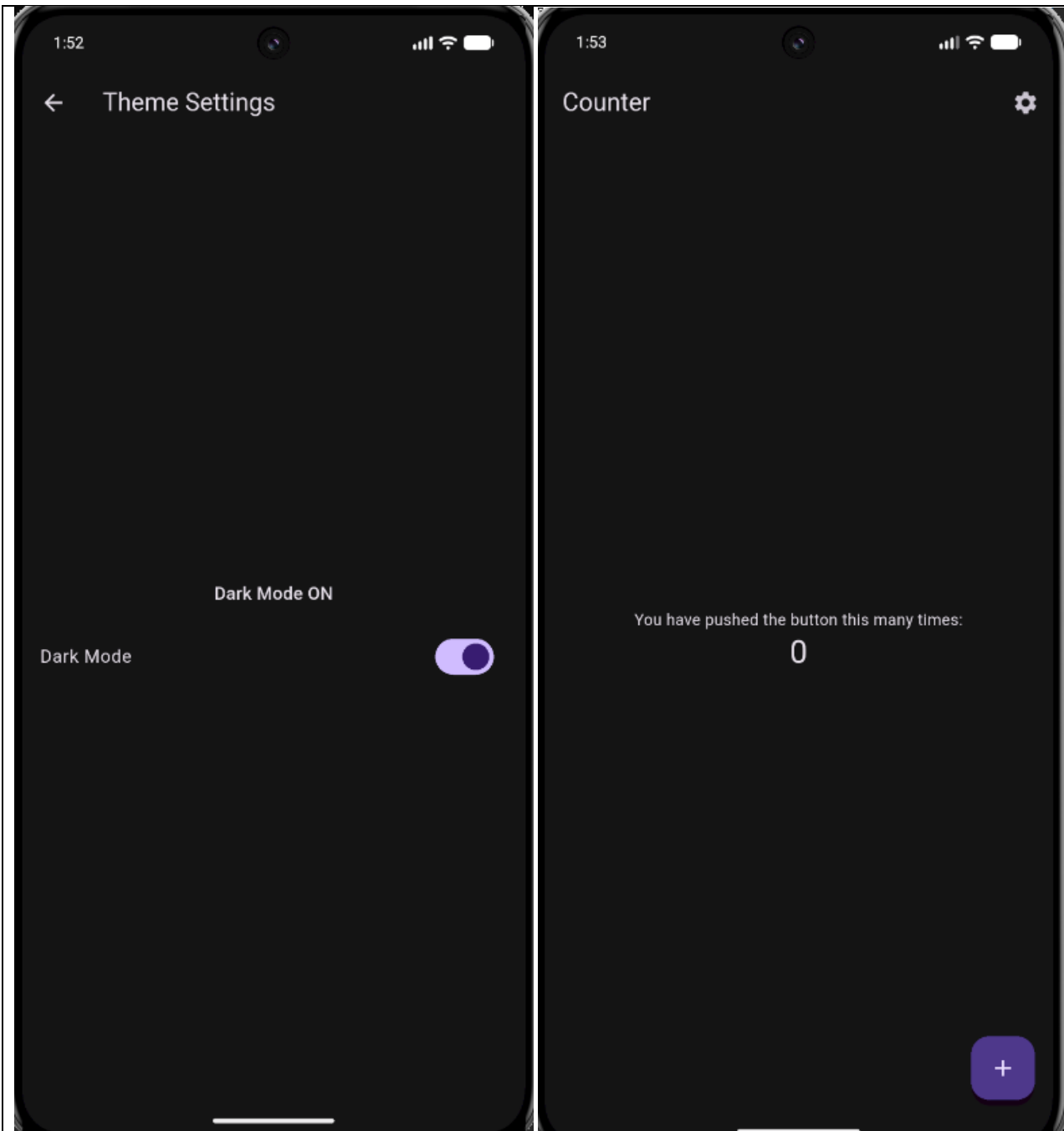
  @override
  Widget build(BuildContext context) {
    final themeModel = Provider.of<ThemeModel>(context);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Theme Settings'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            Text(
              themeModel.isDark ? 'Dark Mode ON' : 'Light Mode ON',
              style: Theme.of(context).textTheme.titleMedium,
            ),
            const SizedBox(height: 16),
            // toggles the theme for the whole app
            SwitchListTile(
              title: const Text('Dark Mode'),
              value: themeModel.isDark,
              onChanged: (_) => themeModel.toggleTheme(),
            ),
          ],
        ),
      ),
    );
  }
}
```



Output – [name of the output]





setState is a simple way to update the UI when something changes in just one widget. I think it's best for small apps or when only a small part of the app needs to change. On the other hand, Provider is used to manage and share data between different widgets more easily. It is more useful for bigger apps because it helps keep the code cleaner and more organized.

Lab Activity Rubric

Name	Jamaine Grace Tuazon
Section	INF231
Date	July 28, 2026

Criteria	Excellent (4 pts)	Good (3 pts)	Fair (2 pts)	Poor (1 pt)	Points
Logic & Code Implementation	Program runs flawlessly with optimal logic flow; all features (placeholders, avatars, widgets for like/comment) are correctly implemented and error-free.	Logic is mostly correct, minor flaws present, or one minor feature missing.	Logic contains noticeable flaws or inefficiencies; two or more features missing.	Code has major logic errors, does not run, or is significantly incomplete.	
Design Accuracy & Consistency	UI matches intended design precisely with proper spacing, alignment, colors, and styles; highly consistent across all components.	Design mostly matches with only minor spacing, alignment, or style issues.	Design deviates noticeably from intended layout; inconsistent styling or incomplete visual elements.	Design is unrecognizable, lacks consistency, or ignores provided specifications.	
Widget Usage & Interactive Elements	Widgets (placeholders, avatars, like/comment buttons) are implemented effectively, enhancing functionality and interactivity.	Most widgets are used correctly with minor implementation or interaction issues.	Widgets are inconsistently used, or two or more interactive features are missing.	Widgets are not implemented, or interactivity is absent.	
Code Structure & Readability	Code is well-structured, modular, and follows best practices; easy to read, maintain, and extend.	Code is mostly organized with minor structural issues or small readability problems.	Code has structural weaknesses or poor formatting that affects readability.	Code is disorganized, hard to read, and does not follow any clear structure.	
Other Comments/ Observations:				Total Score	
				Rating: (Total Score/16) * 100	